

Given a string and a non-negative int n, return a larger string that is n copies of the original string.

string_times('Hi', 2) → 'HiHi'
string_times('Hi', 3) → 'HiHiHi'
string_times('Hi', 1) → 'Hi'

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def string_times(str, n):  
    return str * n
```

Expected	Run		
string_times('Hi', 2) → 'HiHi'	'HiHi'	OK	
string_times('Hi', 3) → 'HiHiHi'	'HiHiHi'	OK	
string_times('Hi', 1) → 'Hi'	'Hi'	OK	
string_times('Hi', 0) → ''	''	OK	
string_times('Hi', 5) → 'HiHiHiHiHi'	'HiHiHiHiHi'	OK	
string_times('Oh Boy!', 2) → 'Oh Boy!Oh Boy!'	'Oh Boy!Oh Boy!'	OK	
string_times('x', 4) → 'xxxx'	'xxxx'	OK	
string_times('', 4) → ''	''	OK	
string_times('code', 2) → 'codecode'	'codecode'	OK	
string_times('code', 3) → 'codecodecode'	'codecodecode'	OK	

 All Correct

Given a string and a non-negative int n, we'll say that the front of the string is the first 3 chars, or whatever is there if the string is less than length 3. Return n copies of the front;

front_times('Chocolate', 2) → 'ChoCho'
front_times('Chocolate', 3) → 'ChoChoCho'
front_times('Abc', 3) → 'AbcAbcAbc'

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def front_times(str, n):  
    front = str[:3]  
    return front * n
```

Expected	Run		
front_times('Chocolate', 2) → 'ChoCho'	'ChoCho'	OK	
front_times('Chocolate', 3) → 'ChoChoCho'	'ChoChoCho'	OK	
front_times('Abc', 3) → 'AbcAbcAbc'	'AbcAbcAbc'	OK	
front_times('Ab', 4) → 'AbAbAbAb'	'AbAbAbAb'	OK	
front_times('A', 4) → 'AAAA'	'AAAA'	OK	
front_times("", 4) → ""	""	OK	
front_times('Abc', 0) → ""	""	OK	

 All Correct

Given a string, return a new string made of every other char starting with the first, so "Hello" yields "Hlo".

string_bits('Hello') → 'Hlo'
string_bits('Hi') → 'H'
string_bits('Heeololeo') → 'Hello'

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def string_bits(str):  
    return str[::2]
```

Expected	Run		
string_bits('Hello') → 'Hlo'	'Hlo'	OK	
string_bits('Hi') → 'H'	'H'	OK	
string_bits('Heeololeo') → 'Hello'	'Hello'	OK	
string_bits('HiHiHi') → 'HHH'	'HHH'	OK	
string_bits('') → ''	''	OK	
string_bits('Greetings') → 'Getns'	'Getns'	OK	
string_bits('Chocoate') → 'Coot'	'Coot'	OK	
string_bits('pi') → 'p'	'p'	OK	
string_bits('Hello Kitten') → 'HloKte'	'HloKte'	OK	
string_bits('hxaxpxpxy') → 'happy'	'happy'	OK	

  All Correct

[next](#) | [chance](#)

Warmup-2 > string_splosion

[prev](#) | [next](#) | [chance](#)

Given a non-empty string like "Code" return a string like "CCoCodCode".

string_splosion('Code') → 'CCoCodCode'
string_splosion('abc') → 'aababc'
string_splosion('ab') → 'aab'

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def string_splosion(str):  
    res = ""  
    for i in range(len(str)):  
        res += str[:i+1]  
    return res
```

Expected	Run		
string_splosion('Code') → 'CCoCodCode'	'CCoCodCode'	OK	
string_splosion('abc') → 'aababc'	'aababc'	OK	
string_splosion('ab') → 'aab'	'aab'	OK	
string_splosion('x') → 'x'	'x'	OK	
string_splosion('fade') → 'ffafadfade'	'ffafadfade'	OK	
string_splosion('There') → 'TThTheTherThere'	'TThTheTherThere'	OK	
string_splosion('Kitten') → 'KKiKitKittKitteKitten'	'KKiKitKittKitteKitten'	OK	
string_splosion('Bye') → 'BByBye'	'BByBye'	OK	
string_splosion('Good') → 'GGoGooGood'	'GGoGooGood'	OK	
string_splosion('Bad') → 'BBaBad'	'BBaBad'	OK	

 All Correct

Warmup-2 > last2

[prev](#) | [next](#) | [chance](#)

Given a string, return the count of the number of times that a substring length 2 appears in the string and also as the last 2 chars of the string, so "hixxxhi" yields 1 (we won't count the end substring).

last2('hixxhi') → 1

last2('xaxxaxaxx') → 1

last2('axxxaaxx') → 2

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def last2(str):  
    if len(str) < 2:  
        return 0  
    last = str[-2:]  
    count = 0  
    for i in range(len(str)-2):  
        if str[i:i+2] == last:  
            count += 1  
    return count
```

Expected

Run

last2('hixxhi') → 1	1	OK	
last2('xaxxaxaxx') → 1	1	OK	
last2('axxxaaxx') → 2	2	OK	
last2('xxaxxaxxaxx') → 3	3	OK	
last2('xaxaxaxx') → 0	0	OK	
last2('xxxx') → 2	2	OK	
last2('13121312') → 1	1	OK	
last2('11212') → 1	1	OK	
last2('13121311') → 0	0	OK	
last2('1717171') → 2	2	OK	
last2('hi') → 0	0	OK	
last2('h') → 0	0	OK	
last2('') → 0	0	OK	



All Correct

Warmup-2 > array_count9

[prev](#) | [next](#) | [chance](#)

Given an array of ints, return the number of 9's in the array.

array_count9([1, 2, 9]) → 1
array_count9([1, 9, 9]) → 2
array_count9([1, 9, 9, 3, 9]) → 3

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def array_count9(nums):  
    return nums.count(9)
```

Expected

Run

array_count9([1, 2, 9]) → 1	1	OK	
array_count9([1, 9, 9]) → 2	2	OK	
array_count9([1, 9, 9, 3, 9]) → 3	3	OK	
array_count9([1, 2, 3]) → 0	0	OK	
array_count9([]) → 0	0	OK	
array_count9([4, 2, 4, 3, 1]) → 0	0	OK	
array_count9([9, 2, 4, 3, 1]) → 1	1	OK	



All Correct

[next](#) | [chance](#)

Warmup-2 > array_front9

[prev](#) | [next](#) | [chance](#)

Given an array of ints, return True if one of the first 4 elements in the array is a 9. The array length may be less than 4.

array_front9([1, 2, 9, 3, 4]) → True

array_front9([1, 2, 3, 4, 9]) → False

array_front9([1, 2, 3, 4, 5]) → False

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def array_front9(nums):
    end = len(nums)
    if end > 4:
        end = 4
    for i in range(end):
        if nums[i] == 9:
            return True
    return False
```

Expected

Run

array_front9([1, 2, 9, 3, 4]) → True	True	OK	
array_front9([1, 2, 3, 4, 9]) → False	False	OK	
array_front9([1, 2, 3, 4, 5]) → False	False	OK	
array_front9([9, 2, 3]) → True	True	OK	
array_front9([1, 9, 9]) → True	True	OK	
array_front9([1, 2, 3]) → False	False	OK	
array_front9([1, 9]) → True	True	OK	
array_front9([5, 5]) → False	False	OK	
array_front9([2]) → False	False	OK	
array_front9([9]) → True	True	OK	
array_front9([]) → False	False	OK	
array_front9([3, 9, 2, 3, 3]) → True	True	OK	



All Correct

[next](#) | [chance](#)

Warmup-2 > array123

[prev](#) | [next](#) | [chance](#)

Given an array of ints, return True if the sequence of numbers 1, 2, 3 appears in the array somewhere.

array123([1, 1, 2, 3, 1]) → True

array123([1, 1, 2, 4, 1]) → False

array123([1, 1, 2, 1, 2, 3]) → True

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def array123(nums):
    for i in range(len(nums)-2):
        if nums[i:i+3] == [1, 2, 3]:
            return True
    return False
```

Expected

Run

array123([1, 1, 2, 3, 1]) → True	True	OK	
array123([1, 1, 2, 4, 1]) → False	False	OK	
array123([1, 1, 2, 1, 2, 3]) → True	True	OK	
array123([1, 1, 2, 1, 2, 1]) → False	False	OK	
array123([1, 2, 3, 1, 2, 3]) → True	True	OK	
array123([1, 2, 3]) → True	True	OK	
array123([1, 1, 1]) → False	False	OK	
array123([1, 2]) → False	False	OK	
array123([1]) → False	False	OK	
array123([]) → False	False	OK	



All Correct

[next](#) | [chance](#)

Warmup-2 > string_match

[prev](#) | [next](#) | [chance](#)

Given 2 strings, a and b, return the number of the positions where they contain the same length 2 substring. So "xxcaazz" and "xxbaaz" yields 3, since the "xx", "aa", and "az" substrings appear in the same place in both strings.

string_match('xxcaazz', 'xxbaaz') → 3
string_match('abc', 'abc') → 2
string_match('abc', 'axc') → 0

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def string_match(a, b):  
    shorter = min(len(a), len(b))  
    count = 0  
    for i in range(shorter - 1):  
        if a[i:i+2] == b[i:i+2]:  
            count += 1  
    return count
```

Expected

Run

string_match('xxcaazz', 'xxbaaz') → 3	3	OK	
string_match('abc', 'abc') → 2	2	OK	
string_match('abc', 'axc') → 0	0	OK	
string_match('hello', 'he') → 1	1	OK	
string_match('he', 'hello') → 1	1	OK	
string_match('h', 'hello') → 0	0	OK	
string_match('', 'hello') → 0	0	OK	
string_match('aabbccdd', 'abbbxxd') → 1	1	OK	
string_match('aaxxaaxx', 'iaxxai') → 3	3	OK	
string_match('iaxxai', 'aaxxaaxx') → 3	3	OK	



All Correct

[next](#) | [chance](#)

Python > Warmup-2